No Man's Sky: The Definitive Technical Architecture of Planetary Generation

This technical whitepaper provides a comprehensive architectural breakdown of the systems governing planetary generation, celestial mechanics, asset distribution, and rendering pipelines within Hello Games' *No Man's Sky* (NMS).

1. The Core Engine Foundation & Deterministic Universe

The entire universe of No Man's Sky is built on mathematical determinism. The game does not read data from a centralized server cluster or cloud database to construct a planet's geometry, climate, or ecology. Instead, it computes the universe on-the-fly using a single master 64-bit seed value.

1.1 The 64-Bit Global Seed Matrix

A 64-bit unsigned integer limits the total number of exploreable planets to exactly 2^{64} (18,446,744,073,709,551,616). This integer serves as the root parameter for a network of cascading Pseudo-Random Number Generators (PRNGs). Because the math is purely functional and stateless, passing the same seed into the generation pipelines will yield identical output vectors down to the coordinate of an individual leaf asset, regardless of the computing platform or execution time.

1.2 The PRNG Cascade and Dependency Hierarchy

Generation relies on a strict deterministic hierarchy, where the output vector of a high-level systemic loop directly instantiates the seed for the subsequent sub-loops:

1. **Galactic Core Seed:** Establishes coordinates for hundreds of billions of individual stellar systems within a single galaxy.
2. **Stellar System Seed:** Decides coordinates for up to 6 celestial bodies, the orbital planes, solar classification (spectral type), faction occupancy, economy matrices, and conflict ratings.
3. **Planetary Core Seed:** Derived directly from the system index position. Governs the global biome archetype, basic radius limits, water table metrics, atmospheric composition, and gravity indices.
4. **Local Detail Seeds:** Broken down into specialized pipelines for voxel terrain deformation fields, flora distribution matrices, fauna skeletal morphing, and regional waypoint placement.

2. Celestial Mechanics and Floating Origin System

To preserve visual coherence and eliminate numerical stability failures over interstellar distances, the engine employs heavily customized geometry and physics frameworks.

2.1 The Illusion of Orbital Mechanics

True orbital mechanics (Newtonian gravity trajectories, active planetary rotation, and genuine planetary revolutions around a fixed coordinate star) are entirely absent from the game. True physics were tested during early pre-release builds but discarded due to negative impacts on player orientation during space-to-ground transit. Instead, planets are completely static relative to one another within a given solar system instance. Day and night cycles are simulated globally by rotating a skybox projection sphere containing the system's star asset around the stationary cluster of planets.

2.2 The Floating Origin Physics System

Standard 32-bit floating-point variables lose numerical precision as object coordinates scale farther away from the origin point (0,0,0). In massive virtual spaces, this leads to geometric jittering, vertex tearing, and broken physics simulation. No Man's Sky resolves this via an aggressive *Floating Origin* pipeline. The player character remains permanently at the relative physics origin (0,0,0) of the immediate scene matrix. As the player flies or walks, the engine translates the universe's absolute coordinate values in reverse. Space objects do not move toward the player; the world data structure shifts around the player's coordinate space.

3. Volumetric Voxel Terrain Generation

Terrain generation does not use standard 2D heightmaps mapped over flat polygons. Instead, all celestial bodies are mapped as true 3D volumetric voxels, enabling caves, arches, floating islands, and direct surface manipulation.

3.1 Mathematical Noise Layers and the Superformula

The basic macromorphology of a planet's surface is calculated through multi-layered noise profiles combined with a modified 3D implementation of Johan Gielis' *Superformula*. The superformula dictates fundamental geometric symmetries and organic contours using the following base polar coordinate equation:

$$r(\varphi) = \left[\left|\cos(m\cdot\varphi/4)/a\right|^{n2} + \left|\sin(m\cdot\varphi/4)/b\right|^{n3} \right]^{-1/n1}$$

By modulating parameters for *m*, *n1*, *n2*, and *n3*, the engine constructs distinct planetary geologies—ranging from rolling plains to vast canyon networks and sheer vertical crags.

This base shape is layered with fractional Brownian motion (fBm), Simplex noise, and Worley noise to generate micro-crevices, ridges, and cavernous chambers.

3.2 Voxel Data Modification and Delta Tracking

Because the terrain is computed voxel-by-voxel, modifying the terrain with the Terrain Manipulator changes the voxel density values. To prevent bloating save files, the engine never stores the full modified planet model. Instead, it maintains a highly compressed *Delta Script*. This script acts as a list of localized exceptions containing the spatial coordinate and radius of voxel addition/subtraction. When a player renders a chunk of terrain, the engine computes the raw mathematical formula first, then overlays the local save file's Delta Script over the output vector before finalizing the vertex buffer.

4. Stellar Spectral Classes & Biome Selection Matrices

The stellar classification of a star system acts as the primary logical filter for the planetary generation engines. The engine reads the spectral designation to alter probabilities for planetary biomes, resources, and exotic variables.

Spectral Class	Primary Star Colour	Warp Drive Required	Dominant Biome Characteristics & Resource Probabilities
Class F & G	Yellow	Standard Hyperdrive	High probability of Lush/Paradise worlds. Standard resource distribution (Copper, Paraffinium). Balanced hazard profiles.
Class M & K	Red	Cadmium Drive	Increased rate of Desert, Scorched, and Empty worlds. Exclusive access to Cadmium. High incidence of anomalous and glitched biomes.
Class E	Green	Emeril Drive	Elevated rate of Toxic and Radioactive worlds. Exclusive access to Emeril. Increased probability of extreme weather conditions.
Class O & B	Blue	Indium Drive	Highest probability of Exotic, Shattered, and Dissonant/Corrupted worlds. Exclusive access to Indium. Extreme hazard levels.

5. Procedural Biology, Skeletal Assembly, & Flora Matrices

Once a planet's physical terrain and biome attributes are established by the seed cascade, the engine initializes the placement matrices for local organic ecosystems.

5.1 Procedural Fauna Morphing Engine

Fauna are not stored as pre-built 3D models. The engine holds a library of core structural archetypes (e.g., Quadruped, Biped, Insectoid, Avian, Aquatic, Megafauna). Each archetype contains a collection of interchangeable geometric components (heads, limbs, tails, horns, wings) along with specified boundary nodes.

- **Skeletal Scaling:** The planetary seed determines bone length, muscular thickness, and overall physical scale, modifying the base animation skeletons dynamically.
- **Texture Projection:** Creature skin, scales, fur, and feather patterns are generated procedurally by applying multi-layered noise masks directly over the 3D meshes, avoiding fixed 2D texture wraps.

5.2 Mathematical Asset Suppressors for Flora

To maintain visual immersion, trees, rocks, and hazard plants are distributed based on mathematical vector checks against the local voxel field:

- **Slope Invalidation:** The engine calculates the vertex normal vector of the generated terrain. If a cliff surface exceeds a 45-degree angle, the tree placement engine suppresses spawning to prevent assets from clipping out horizontally.
- **Altimetric Constraints:** Flora density is mapped against the local water table data. Certain plant classes are entirely restricted to low-altitude coastal zones, while others only spawn above specific mountain altitudes.

6. Seamless Space-to-Ground Rendering Pipeline

Transitioning from deep space to a planet's surface without loading screens requires an aggressive level-of-detail (LOD) caching strategy that utilizes asynchronous GPU threads.

6.1 Asynchronous Quadtree LOD Management

From space, a planet is rendered as a simple, low-polygon sphere wrapped in a low-resolution texture map derived from the global planetary seed. As the player’s spaceship approaches the planet's surface, the engine divides the planetary sphere into an array of patches using a *Quadtree* system.

The quadtree continuously evaluates the player's vector distance and velocity. As a patch nears the camera, it splits into four high-resolution sub-patches. This process repeats recursively, streaming data in real-time until the engine swaps the low-res mesh for full 3D voxel terrain blocks right as the atmosphere boundary is crossed.

6.2 Voxel Meshing via Dual Contouring

To transform raw volumetric voxel data blocks into smooth, renderable surfaces for the graphics card, the engine runs a real-time *Dual Contouring* algorithm. Dual contouring evaluates voxel density intersections along a 3D grid, preserving sharp geometric features,

mountain ridges, and steep inclines far better than older algorithms like Marching Cubes. This calculation runs on background CPU threads, feeding finalized index and vertex buffers directly into the GPU pipeline just in time as the camera moves through the space.

This is for informational purposes only. For medical advice or diagnosis, consult a professional. AI responses may include mistakes.